

UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

Práctica Guiada de PowerShell

Automatización de datos para la toma de decisiones de negocio

Caso práctico: catálogo de películas (IMDb) · lab_03_IMDb

1. Contexto y objetivo de negocio

El equipo de Distribución de Contenidos de una plataforma de streaming necesita planear la carga de trabajo del próximo trimestre. Cada película del catálogo debe entregarse a los usuarios finales en distintos formatos (4K, HD, SD y Móvil): a esas copias adicionales por formato se les llama réplicas.

Como criterio de negocio, se asume que entre más votos acumula una película en IMDb, mayor es su popularidad y, por lo tanto, mayor prioridad y número de réplicas deberá preparar el equipo de Operaciones.

Tu misión

Usando PowerShell, obtener el Top 10 de películas con mayor número de votos (No_of_Votes) a partir de un catálogo público de 1000 películas, y generar un reporte en CSV que el equipo de Operaciones usará para planear las réplicas.

Este es solo un ejemplo de aplicación de negocio: los mismos comandos que usarás aquí sirven para responder muchas otras preguntas (por ejemplo, qué productos reabastecer primero, qué clientes contactar antes, o qué reportes generar de forma automática).

2. Pseudocódigo general

Antes de tocar el dataset, cada alumno debe dejar preparados dos elementos: su identidad (para nombrar sus evidencias) y su bitácora de trabajo. Esta es la configuración inicial del proyecto — Paso 0:

- Declarar tus datos de identidad (nombre y número de cuenta) como variables.
- Resolver la ruta del Escritorio de forma portable.
- Iniciar la bitácora (transcript) que grabará toda tu sesión.

Con el entorno ya configurado, esta es la ruta completa del reto de datos:

1. Crear, dentro del Escritorio, la carpeta de trabajo del proyecto: lab_03_IMDb.
2. Descargar el archivo CSV del catálogo de películas.
3. Confirmar que el archivo se descargó correctamente.
4. Cargar el archivo CSV en una variable, como tabla de datos.
5. Explorar las columnas (propiedades) disponibles en los datos.
6. Ordenar las películas por número de votos, de mayor a menor.
7. Quedarse solo con las primeras 10 películas (Top 10).
8. Quedarse solo con las columnas necesarias para el reporte final.
9. Exportar el resultado a un nuevo archivo CSV (el entregable).
10. Comprimir el entregable en un archivo .zip para enviarlo al equipo de Operaciones.
11. Detener la bitácora antes de reunir tu evidencia para la entrega.

 Cómo usar esta guía

El Paso 0 (configuración) se resuelve con código ya dado, que debes copiar y ajustar con tus propios datos. Los Pasos 1 al 10 (el reto de datos) se desarrollan con el o los cmdlets a investigar, pistas y una salida esperada, para que tú construyas cada comando.

3. Desarrollo de la práctica

Trabaja los pasos en orden. Abre una terminal de PowerShell y comienza siempre por el Paso 0, antes de tocar cualquier dato.

Paso 0. Configurar identidad y bitácora del proyecto

 Objetivo del paso

Para garantizar que cada alumno generó apropiadamente la práctica, toda tu sesión de PowerShell (comandos y salidas) quedará grabada en un archivo de bitácora (.log), identificado con tu número de cuenta. Ese archivo es, junto con el .zip del proyecto, la evidencia que entregarás al profesor.

Todos los alumnos trabajarán sobre una carpeta con el mismo nombre, lab_03_IMDb, ubicada directamente en el Escritorio. Tu bitácora se guardará también en el Escritorio, al mismo nivel que esa carpeta (es decir, como “vecina” de lab_03_IMDb, no dentro de ella), y su nombre dependerá de tu propio número de cuenta.

Copia los siguientes tres bloques de código en tu terminal, en este orden, y ajusta únicamente lo que se indica.

 1) Declara tus datos de identidad primero para usarlos en el nombre del archivo.

```
$env:NOMBRE_APELLIDOS = "NOMBRE_APELLIDO1_APELLIDO2"
$env:NUM_CUENTA = "12345678"
```

 Ajusta antes de ejecutar

Sustituye NOMBRE_APELLIDO1_APELLIDO2 por tus propios datos (sin espacios ni acentos) y 12345678 por tu número de cuenta real. Estas dos líneas deben ejecutarse cada vez que abras una nueva terminal para trabajar en la práctica.

 2) Resuelve la carpeta del Escritorio de forma portable.

```
# En Windows, el nombre real puede cambiar por idioma o perfil; si no existe, usa la carpeta actual.
$logDir = [Environment]::GetFolderPath('Desktop')
if (-not $logDir -or -not (Test-Path -LiteralPath $logDir)) {
    $logDir = $PWD.Path
}
```

 Por qué esto importa

El nombre de la carpeta del Escritorio no siempre es literalmente “Desktop” (depende del idioma de Windows y del perfil de usuario). `[Environment]::GetFolderPath('Desktop')` te da la ruta correcta sin que tengas que adivinarla ni escribirla “a mano”. Reutilizarás `$logDir` también en el Paso 1, para crear ahí mismo tu carpeta `lab_03_IMDb`.

3) Genera la ruta del archivo de evidencia e inicia la grabación.

```
# Genera la ruta del archivo de evidencia sin asumir una carpeta fija ni idioma.
$logPath = Join-Path $logDir "practica_SO_$env:NUM_CUENTA.log"

# Inicia la grabación de todo lo que aparezca en la consola (comandos y salidas)
Start-Transcript -Path $logPath
```

Salida esperada (para autoconfirmar):

```
Transcript started, output file is
C:\Users\alumno\Desktop\practica_SO_12345678.log
```

 A partir de este momento, todo lo que escribas y toda la salida que aparezca en pantalla quedará registrada en tu archivo `.log`. No cierres la ventana de PowerShell hasta terminar la práctica y detener la bitácora (Paso 11).

Paso 1. Preparar el entorno de trabajo

Objetivo del paso

Todos los alumnos del grupo trabajarán sobre una carpeta con el mismo nombre —para que sea fácil de identificar en la revisión—: `lab_03_IMDb`, ubicada directamente dentro de tu Escritorio.

En vez de escribir la ruta del Escritorio “a mano”, reutiliza la variable `$logDir` que ya resolviste en el Paso 0: así tu comando funciona igual sin importar el nombre de usuario o el idioma de Windows.

Cmdlet(s) a investigar

- **New-Item** crea un elemento nuevo (carpeta o archivo) en el sistema de archivos.
- **Join-Path** combina dos fragmentos de ruta en uno solo, de forma segura y portable.

Parámetros / banderas que te pueden servir:

- **-Path** ruta donde se creará el elemento (puedes construirla con `Join-Path $logDir "lab_03_IMDb"`).
- **-ItemType Directory** indica que el elemento a crear es una carpeta, no un archivo.

Tips

Guarda la ruta resultante en una variable, por ejemplo `$projectDir`, para reutilizarla en los siguientes pasos en vez de volver a escribirla.

El nombre de la carpeta debe ser exactamente `lab_03_IMDb` (en minúsculas, sin espacios) para todos los alumnos.

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

✓ Salida esperada (para autoconfirmar):

Directorio: C:\Users\alumno\Desktop

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d----	21/08/2026 10:15		lab_03_IMDb

 El formato exacto de fecha/hora puede variar según la configuración regional de tu equipo; lo importante es que la carpeta lab_03_IMDb aparezca listada dentro de tu Escritorio.

Paso 2. Descargar el dataset de películas

Objetivo del paso

El área de Análisis de Datos identificó un catálogo público con 1000 películas (título, año, género, calificación, votos, recaudación, entre otros datos) en formato CSV, de tamaño manejable. Tu tarea es descargarlo dentro de la carpeta lab_03_IMDb creada en el Paso 1.

Fuente sugerida para la práctica (archivo espejo público, sin necesidad de crear cuenta):

https://raw.githubusercontent.com/WictorDalbosco/Computer-Visualization/main/imdb_top_1000.csv

Fuente original del dataset (requiere cuenta gratuita, opcional para consultarlo desde el navegador):

Kaggle — “IMDb Dataset of Top 1000 Movies and TV Shows”.

Cmdlet(s) a investigar

- **Invoke-WebRequest** realiza una solicitud web y permite guardar el contenido descargado en disco.

Parámetros / banderas que te pueden servir:

- **-Uri** dirección completa (https://...) del archivo a descargar.
- **-OutFile** ruta y nombre con el que se guardará el archivo localmente (incluye la extensión .csv).

Tips

Guarda el archivo dentro de \$projectDir (la carpeta lab_03_IMDb del Paso 1) y consérvalo con el nombre imdb_top_1000.csv; lo necesitarás igual en los siguientes pasos.

Si la descarga tarda demasiado o falla, confirma que copiaste la URL directa al archivo (la que empieza con raw.githubusercontent.com) y no la página de vista previa de GitHub.

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

✓ Salida esperada (para autoconfirmar):

(si todo salió bien, el comando no imprime nada en pantalla; eso es normal)

i Si ves un error como “No se pudo resolver el nombre remoto” o “403”, revisa tu conexión a internet o la URL utilizada. Confirma el resultado real en el Paso 3.

Paso 3. Confirmar que la descarga fue exitosa

🎯 Objetivo del paso

Antes de continuar, valida que el archivo realmente se guardó en disco y que no está vacío o corrupto.

🔧 Cmdlet(s) a investigar

- **Test-Path** verifica si una ruta (archivo o carpeta) existe; devuelve \$true o \$false.
- **Get-Item** obtiene el objeto de archivo, con sus propiedades (incluido su tamaño).

Parámetros / banderas que te pueden servir:

- **-Path** ruta del archivo a verificar / consultar.

💡 Tip

Get-Item también te muestra la propiedad Length (tamaño en bytes) del archivo. Si es 0, algo falló en la descarga.

🔧 Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

✓ Salida esperada (para autoconfirmar):

```
True
```

```
Directorio: C:\Users\alumno\Desktop\lab_03_IMDb
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a---	21/08/2026 10:20	~188000	imdb_top_1000.csv

i El tamaño exacto (Length) puede variar ligeramente según la fuente, pero debe rondar los 180–200 KB; nunca 0 bytes.

Paso 4. Cargar el dataset como tabla de datos

Objetivo del paso

Para poder manipular la información con comandos de PowerShell (ordenar, filtrar, seleccionar columnas), primero necesitas convertir el archivo de texto plano CSV en objetos que PowerShell entienda, y guardarlos en una variable para reutilizarlos en los siguientes pasos.

Cmdlet(s) a investigar

- **Import-Csv** lee un archivo CSV y convierte cada fila en un objeto, donde cada columna del encabezado se vuelve una propiedad.

Parámetros / banderas que te pueden servir:

- **-Path** ruta del archivo CSV a importar.
- **-Delimiter** carácter separador de columnas (por defecto es la coma; úsalo solo si el archivo no se lee bien).

Tips

Guarda el resultado en una variable, por ejemplo `$peliculas = Import-Csv ...`, en vez de dejar que se imprima todo en pantalla (son 1000 filas).

Recuerda: `Import-Csv` siempre lee todos los valores como texto (string), aunque parezcan números. Esto será importante más adelante.

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

Salida esperada (para autoconfirmar):

```
PS> $peliculas.Count  
1000
```

```
PS> $peliculas[0].Series_Title  
The Shawshank Redemption
```

 Si `$peliculas.Count` no marca 1000, revisa que `-Path` apunte exactamente al archivo del Paso 2 y que la descarga se haya completado.

Paso 5. Explorar las columnas disponibles

Objetivo del paso

Antes de decidir por cuál columna ordenar o cuáles conservar en el reporte, necesitas conocer con qué información cuentas realmente.

Cmdlet(s) a investigar

- **Get-Member** muestra las propiedades (y métodos) de los objetos que recibe por la tubería (pipeline).

Parámetros / banderas que te pueden servir:

- **-MemberType NoteProperty** filtra para mostrar solo las propiedades de datos (las columnas), sin métodos ni otros miembros.

Tip

Envía tu variable de películas hacia Get-Member usando el operador de tubería (|). No necesitas volver a importar el archivo.

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

Salida esperada (para autoconfirmar):

```
TypeName: System.Management.Automation.PSCustomObject
```

Name	MemberType	Definition
----	-----	-----
Certificate	NoteProperty	...
Director	NoteProperty	...
Genre	NoteProperty	...
Gross	NoteProperty	...
IMDB_Rating	NoteProperty	...
Meta_score	NoteProperty	...
No_of_Votes	NoteProperty	...
Overview	NoteProperty	...
Poster_Link	NoteProperty	...
Released_Year	NoteProperty	...
Runtime	NoteProperty	...
Series_Title	NoteProperty	...
Star1 / Star2 / Star3 / Star4	NoteProperty	...

 En total deberías identificar 16 propiedades. Confirma que entre ellas aparece una columna relacionada con el número de votos: la necesitarás en el Paso 6.

Paso 6. Ordenar las películas por demanda (votos)

Objetivo del paso

El criterio de negocio para priorizar réplicas es la demanda: entre más votos tiene una película en IMDb, más popular es y más formatos de entrega (4K, HD, SD, Móvil) deberá preparar el equipo de Operaciones. Ordena el catálogo completo de mayor a menor número de votos.

Cmdlet(s) a investigar

- **Sort-Object** ordena objetos según el valor de una o varias propiedades.

Parámetros / banderas que te pueden servir:

- **-Property** propiedad (columna) por la que se ordenará.
- **-Descending** invierte el orden, de mayor a menor.

Tips

 Import-Csv devuelve todos los valores como texto (string), no como número. Si ordenas directamente por la propiedad de votos “tal cual”, PowerShell comparará los valores como texto (letra por letra), no como cantidad.

Investiga cómo usar una propiedad calculada (calculated property) dentro de -Property para convertir el valor a número entero (por ejemplo, con [int]) antes de ordenar.

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

Salida esperada (para autoconfirmar):

```
PS> $ordenadas = ...
PS> $ordenadas[0].No_of_Votes
2343110
PS> $ordenadas[-1].No_of_Votes
25088
```

 Lo esencial es que el primer registro tenga un número de votos mucho mayor (varios millones) que el último. Si ves lo contrario, o números “pequeños” arriba, tu ordenamiento se hizo como texto: revisa la conversión numérica.

Paso 7. Quedarse con el Top 10

Objetivo del paso

De la lista ya ordenada por demanda, el equipo de Operaciones solo necesita trabajar con las 10 películas que requerirán más réplicas.

Cmdlet(s) a investigar

- **Select-Object** selecciona un subconjunto de objetos y/o de propiedades.

Parámetros / banderas que te pueden servir:

- **-First** número de objetos a conservar, contando desde el principio de la colección.

Tip

Aplica este comando sobre el resultado ya ordenado del Paso 6 (puedes encadenarlo con `|` o reutilizar la variable).

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

Salida esperada (para autoconfirmar):

```
PS> ($top10 | Measure-Object).Count
10
```

 Si el conteo no da exactamente 10, revisa el valor que usaste después de `-First`.

Paso 8. Quedarse solo con las columnas del reporte

Objetivo del paso

El equipo de Operaciones no necesita las 16 columnas originales; solo requiere la información relevante para planear las réplicas: título, año, género y número de votos (puedes agregar alguna columna adicional si lo consideras útil).

Cmdlet(s) a investigar

- **Select-Object** también permite elegir qué propiedades (columnas) conservar de cada objeto.

Parámetros / banderas que te pueden servir:

- **-Property** lista de una o varias propiedades a conservar, separadas por comas.

Tip

Esta vez el mismo cmdlet del Paso 7 se usa con un parámetro distinto: en vez de limitar cuántas filas, limita cuántas columnas.

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

Salida esperada (para autoconfirmar):

Series_Title	Released_Year	Genre	No_of_Votes
--------------	---------------	-------	-------------

```

-----
The Shawshank Redemption 1994          Drama  2343110
...                          ...          ...    ...

```

 Este es solo un ejemplo del formato esperado (una fila real de tu dataset), no indica cuál película debe quedar en qué posición de tu Top 10 final.

Paso 9. Exportar el reporte entregable

Objetivo del paso

El resultado debe entregarse como un archivo CSV nuevo, independiente del original, para que el equipo de Operaciones pueda abrirlo directamente en Excel.

Cmdlet(s) a investigar

- **Export-Csv** guarda objetos de PowerShell como un archivo CSV.

Parámetros / banderas que te pueden servir:

- **-Path** ruta y nombre del archivo CSV de salida (por ejemplo, Top10_Replicas.csv, dentro de \$projectDir).
- **-NoTypeInfo** evita que se agregue una línea adicional con el tipo de objeto .NET al inicio del archivo.
- **-Encoding** codificación de caracteres del archivo (UTF8 es una opción segura para acentos y símbolos).

Tip

Si abres el CSV en Excel y la primera línea dice algo como “#TYPE ...”, olvidaste -NoTypeInfo.

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

Salida esperada (para autoconfirmar):

(el comando no imprime nada; el archivo se crea en disco)

```
PS> (Import-Csv .\Top10_Replicas.csv).Count
10
```

 Vuelve a importar tu propio archivo exportado como forma rápida de comprobar que quedó bien formado y con 10 filas.

Paso 10. Comprimir el entregable

Objetivo del paso

Para enviar el reporte por correo de forma más práctica, el equipo de Operaciones pide que el entregable se envíe comprimido.

Cmdlet(s) a investigar

- **Compress-Archive** comprime uno o varios archivos/carpetas en un archivo .zip.

Parámetros / banderas que te pueden servir:

- **-Path** archivo(s) o carpeta a comprimir (por ejemplo, todo \$projectDir).
- **-DestinationPath** ruta y nombre del archivo .zip resultante.

Tip

Puedes comprimir solo el CSV entregable, o toda la carpeta lab_03_IMDb si quieres incluir evidencia adicional. Este .zip es una de las dos evidencias que enviarás al profesor (ve la sección de Entrega).

Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

Salida esperada (para autoconfirmar):

```
Directorio: C:\Users\alumno\Desktop\lab_03_IMDb
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a---	21/08/2026 11:05	~1200	Top10_Replicas.zip

Paso 11. Detener la bitácora antes de reunir tu evidencia

Objetivo del paso

Start-Transcript sigue grabando hasta que se detiene de forma explícita. Si no la detienes, el archivo .log puede quedar incompleto o bloqueado cuando intentes adjuntarlo a tu correo de entrega.

Cmdlet a investigar

- **Stop-Transcript** detiene la grabación iniciada con Start-Transcript y cierra el archivo .log.

Tip

Ejecuta este comando como lo último de tu sesión, después de haber comprimido tu entregable en el Paso 10.

 Tu turno — construye el comando:

```
PS ... \Desktop\lab_03_IMDb>
```

```
PS ... \Desktop\lab_03_IMDb>
```

 Salida esperada (para autoconfirmar):

```
Transcript stopped, output file is  
C:\Users\alumno\Desktop\practica_S0_12345678.log
```

 Verifica con `Get-Item` la ruta de tu archivo `.log` para confirmar que su tamaño (`Length`) no es 0 bytes antes de adjuntarlo.

4. Reto extra (opcional)

Si terminaste antes de tiempo, investiga cómo usar `Measure-Object` con el parámetro `-Sum` para calcular qué porcentaje del total de votos del catálogo completo representan las 10 películas de tu reporte final. Preséntalo como un solo número (por ejemplo, con dos decimales).

5. Lista de verificación (autoevaluación)

- ☐ Declaré `$env:NOMBRE_APELLIDOS` y `$env:NUM_CUENTA` con mis datos reales antes de iniciar.
- ☐ Resolví `$logDir` con `[Environment]::GetFolderPath('Desktop')` y confirmé que la ruta existe.
- ☐ Inicié `Start-Transcript` antes de crear cualquier carpeta o descargar cualquier archivo.
- ☐ Mi carpeta de proyecto se llama exactamente `lab_03_IMDb` y está directamente en mi Escritorio.
- ☐ El archivo CSV descargado no está vacío (`Length` mayor a 0).
- ☐ `$peliculas.Count` muestra 1000 después de `Import-Csv`.
- ☐ Identifiqué correctamente el nombre exacto de la columna de votos con `Get-Member`.
- ☐ Mi ordenamiento por votos es numérico, no alfabético (usé una conversión a número).
- ☐ Mi reporte final tiene exactamente 10 filas y solo las columnas relevantes.
- ☐ Exporté el archivo con `Export-Csv -NoTypeInfo`.
- ☐ Comprimí mi entregable en un `.zip`.
- ☐ Detuve la bitácora con `Stop-Transcript` antes de reunir mi evidencia para enviarla.

6. Entrega de la práctica

La entrega de esta práctica se realiza por correo electrónico al profesor, adjuntando dos archivos como evidencia:

- El archivo .zip del proyecto, generado en el Paso 10 (contiene, al menos, tu reporte Top10_Replicas.csv).
- El archivo .log de tu bitácora, ubicado en tu Escritorio con el nombre practica_SO_<tu número de cuenta>.log, generado en el Paso 0 y cerrado en el Paso 11.

**Antes de enviar**

Confirma que ejecutaste Stop-Transcript (Paso 11): un archivo .log sin cerrar puede aparecer incompleto o con un tamaño de 0 bytes para el profesor.

Verifica que ambos archivos correspondan a tu número de cuenta real, ya declarado en \$env:NUM_CUENTA.

Sugerencia de asunto del correo: “Práctica PowerShell IMDb — <Nombre Apellidos> — <Número de cuenta>”.

Sin ambas evidencias (el .zip del proyecto y el .log de la bitácora), la práctica se considera incompleta.

Material elaborado para fines didácticos. Fuente de datos: catálogo público “IMDb Top 1000 Movies”. UAEMEX.